

MEC106A Lab 5

Vision-Guided Pick and Place — Complete Code Guide

UR7e Robot · ROS2 · YOLO · SAM2 · 6D Pose Estimation

This document provides an in-depth reference for every node, function, topic, and call chain in the Lab 5 codebase. It covers the full pipeline from camera input to robot execution, including the 6D pose estimation offline stage, the real-time detection loop, the arm-orbiting inspection phase, the pick-and-place state machine, and the commander orchestration layer.

Table of Contents

1. System Architecture Overview
2. ROS2 Node Inventory
3. Complete Topic & Service Map
4. Node Deep-Dive: DetectionNode (perception)
 - 4.1 YOLODetector helper class
 - 4.2 pixel_to_world helpers
5. Node Deep-Dive: ConstantTransformPublisher (static_tf_transform.py)
6. Node Deep-Dive: IKPlanner (ik.py)
7. Node Deep-Dive: ArmCircler (arm_circler.py)
8. Node Deep-Dive: UR7e_CubeGrasp (main.py)
 - 8.1 Subscriptions and publishers
 - 8.2 Job queue mechanism
 - 8.3 Two-phase centroid refinement
 - 8.4 Task-list auto-execution
 - 8.5 State machine: busy / refining / going_home
 - 8.6 Pick sequence — full call chain
9. Node Deep-Dive: Commander (commander.py)
10. 6D Pose Estimation Pipeline
 - 10.1 run.py — entry point
 - 10.2 pose_estimator.py — five sub-components
 - 10.3 RANSAC + weighted DLT triangulation
 - 10.4 constants.py — configuration
11. End-to-End Pipeline Call Chain
12. Per-Class Grasp Offsets Reference
13. Key Design Patterns
14. Known Issues and Watch-outs

1. System Architecture Overview

The Lab 5 system is a fully autonomous pick-and-place pipeline for a UR7e robot arm. It integrates a RealSense RGBD camera, a YOLO object detector, SAM2 instance segmentation, and a custom 6D pose estimator to locate fruit objects on a table and place them onto a plate.

The system runs in two distinct phases:

- **Inspection Phase (offline data collection):** The ArmCircler node drives the arm around the plate in a 2-row orbit (40 waypoints total), capturing images and recording end-effector poses. When the orbit completes, the Commander node triggers YOLO+SAM2 segmentation followed by multi-view RANSAC triangulation to produce 3D object positions in the robot's base_link frame.
- **Execution Phase (pick and place):** The Commander publishes a JSON task list and signals main.py to start. UR7e_CubeGrasp reads the tasks, moves to pre-pregrasp above each 6D estimate, waits for live YOLO to refine the exact centroid, then executes a full pick-place trajectory through IK + MoveIt.

High-Level Data Flow

```
RealSense Camera
■■ /camera/camera/color/image_raw ■■■■ DetectionNode.image_callback()
■■ /camera/camera/depth/color/points ■■■■ DetectionNode.cloud_callback()
■■ /camera/camera/color/camera_info ■■■■ DetectionNode.camera_info_callback()
■
/detected_class ■ publish on each frame
/detected_pick_point ■■■■ (best non-plate detection)
/detected_plate_point ■■■■ (plate always published)

ArmCircler
■■ subscribes: /detected_plate_point (triggers orbit once)
■■ subscribes: /joint_states (seeds IK)
■■ saves: captured_images/captured_image_{1..40}.jpg
■■ saves: poses.npy [x,y,z,qx,qy,qz,qw] per waypoint
■■ publishes: /orbit_done Bool(True) ■■■■ Commander

Commander (separate terminal, stdin loop)
■■ on /orbit_done: subprocess → segment_batch.py
■■ on /orbit_done: subprocess → 6D_poses/run.py
■■ loads results/object_poses.npy, prints to terminal
■■ user types index/name → publishes /set_target_class + /target_drop_point
■■ user presses Enter → publishes /start_pick_place

UR7e_CubeGrasp (main.py)
■■ on /start_pick_place: _go_home() then auto-run task list
■■ on /target_drop_point: arm pick triggered (live detection path)
■■ on /detected_pick_point: cube_callback() → two-phase IK
■■ IKPlanner → /compute_ik (MoveIt) → /plan_kinematic_path
→ /scaled_joint_trajectory_controller (action)
→ /toggle_gripper (service)
```

2. ROS2 Node Inventory

Six Python ROS2 nodes constitute the system, split across two packages: **perception** and **planning**.

Node Name	Package / File	Launched By	Role
detection_node	perception/detection_node.py	lab5_bringup.launch.py	Real-time YOLO detection; 3D centroid extraction; publishes pick + plate points
ik_planner	planning/ik.py	lab5_bringup.launch.py	Movelt IK wrapper — compute_ik() and plan_to_joints()
constant_tf_publisher	planning/static_tf_transform.py	lab5_bringup.launch.py	Broadcasts static wrist_3_link → camera_link TF at startup
arm_circler	planning/arm_circler.py	lab5_bringup.launch.py (optional)	Orbits arm around plate; captures images + poses; signals orbit_done
cube_grasp	planning/main.py	lab5_bringup.launch.py	Top-level pick-and-place state machine; owns job queue and busy semaphore
commander	planning/commander.py	ros2 run planning commander	CLI orchestrator; runs pose pipeline subprocesses; user interaction loop

Launch Configuration

The launch file **src/planning/launch/lab5_bringup.launch.py** starts all nodes except the camera and commander. Key launch arguments:

Argument	Default	Effect
robot_ip	192.168.1.102	UR7e controller IP for Movelt driver
target_class	""	Initial YOLO target class (empty = highest confidence)
skip_circler	false	Skip orbit phase; UR7e_CubeGrasp publishes /orbit_done immediately (TRANSIENT_LOCAL QoS so Commander receives it late-join)
launch_rviz	true	Launch RViz with Movelt config
shutdown_on_exit	true	Kill all nodes if any process exits

3. Complete Topic and Service Map

Every topic, service, and action used in the system is listed below. The *Direction* column shows which node produces (→) and which node consumes it.

Topic / Service / Action	Type	Direction	Purpose
/camera/camera/color/image_raw	sensor_msgs/Image	Camera → DetectionNode, ArmCircler, main.py	RGB frames; YOLO runs on each frame
/camera/camera/depth/color/points	sensor_msgs/PointCloud2	Camera → DetectionNode	Depth point cloud for 3D centroid extraction
/camera/camera/color/camera_info	sensor_msgs/CameraInfo	Camera → DetectionNode	Intrinsics (fx, fy, cx, cy) for deprojection
/detected_pick_point	geometry_msgs/PointStamped	DetectionNode → main.py	3D centroid of best non-plate object in base_link
/detected_class	std_msgs/String	DetectionNode → main.py	YOLO class label corresponding to the pick point
/detected_plate_point	geometry_msgs/PointStamped	DetectionNode → main.py, ArmCircler, Commander	Plate centroid in base_link; always published regardless of target_class
/set_target_class	std_msgs/String	Commander → DetectionNode	Runtime switch of which YOLO class to target
/target_drop_point	geometry_msgs/PointStamped	Commander → main.py	Drop position + arms main.py for one live-detection pick (_pick_triggered=True)
/start_pick_place	std_msgs/Bool	Commander / main.py → main.py	Activates pick-place (sets pick_place_enabled=True) and calls _go_home()
/orbit_done	std_msgs/Bool	ArmCircler / main.py → Commander	Signals orbit complete; TRANSIENT_LOCAL QoS so late-joiners get it
/pick_task_list	std_msgs/String (JSON)	Commander → main.py	Pre-computed ordered task list [{object_name, pick:[x,y,z], drop:[x,y,z]}, ...]
/joint_states	sensor_msgs/JointState	Robot → main.py, ArmCircler	Live joint angles; seeds IK calls + triggers initial _go_home()
/scaled_joint_trajectory_controller/follow_joint_trajectory	control_msgs/FollowJointTrajectory (Action)	main.py, ArmCircler → Controller	Executes planned joint trajectory on the physical robot
/toggle_gripper	std_srvs/Trigger (Service)	main.py → Gripper driver	Toggle gripper open/closed
/compute_ik	moveit_msgs/GetPositionIK (Service)	IKPlanner → MoveIt	Compute joint angles for a target Cartesian pose
/plan_kinematic_path	moveit_msgs/GetMotionPlan (Service)	IKPlanner → MoveIt	Plan collision-free trajectory to target joint config (RRTConnect)

QoS Profiles

Most topics use default (KEEP_LAST, VOLATILE) QoS. The following use **TRANSIENT_LOCAL** (latched) so that late-joining subscribers receive the last published message:

- **/orbit_done** — published by ArmCircler (or main.py when skip_circler=true); subscribed by Commander. Ensures Commander receives the signal even if it starts after the message is published.

4. Node Deep-Dive: DetectionNode

File: src/perception/perception/detection_node.py | **Node name:** detection_node

DetectionNode is the primary perception node. It runs at camera framerate (30 Hz) and produces 3D object centroids in the robot's base_link frame. It is the only node that reads the live camera stream during normal operation.

Subscriptions

- **/camera/camera/color/image_raw** → image_callback(msg) — main processing callback; runs YOLO on every frame
- **/camera/camera/depth/color/points** → cloud_callback(msg) — buffers latest PointCloud2
- **/camera/camera/color/camera_info** → camera_info_callback(msg) — buffers intrinsics
- **/set_target_class** → _set_target_class_cb(msg) — updates self.target_class at runtime

Publishers

- **/detected_pick_point** (PointStamped) — 3D centroid of highest-confidence non-plate object in base_link
- **/detected_plate_point** (PointStamped) — 3D centroid of plate; always published regardless of target_class filter
- **/detected_class** (String) — class label for the published pick point

image_callback() — Core Processing Loop

Called on every RGB frame. Performs the following steps:

1. Convert ROS Image → OpenCV BGR via CvBridge
2. Call YOLODetector.detect(cv_image) → list of detections
3. For each detection:
 - a. Skip if target_class is set AND class_name != target_class AND class_name != "plate"
(plate is always processed regardless of target_class)
 - b. Call get_centroid_from_cloud_bbox(cloud, camera_info, bbox)
→ reprojects PointCloud2 into image coords, averages XYZ inside bbox
 - c. Call transform_point(tf_buffer, pt_cam, "base_link")
→ TF2 lookup to convert camera_frame → base_link
 - d. If class == "plate": publish on /detected_plate_point, store as self.plate_position, continue
 - e. Plate-radius filter: if 2D dist from plate_position < 0.14m → skip (object is on/near plate)
 - f. Keep track of best_pick_point by highest confidence score
4. After loop: publish best_pick_point on /detected_pick_point + /detected_class
5. If show_image=True: draw YOLO bounding boxes on frame, display via cv2.imshow()

Plate-Radius Exclusion Filter

Once the plate centroid is known, any detection whose 2D distance from the plate (in base_link XY) is less than **self.plate_radius = 0.14 m** is silently dropped. This prevents the robot from trying to pick objects that are already on the plate. The threshold is configurable in code.

Watchdog Timer

A 5-second repeating timer (`_watchdog_cb`) warns if no RGB frames or point cloud messages have been received. It cancels itself once both streams are flowing. This helps diagnose camera startup issues.

Declare Parameters

Parameter	Default	Description
<code>image_topic</code>	<code>/camera/camera/color/image_raw</code>	RGB image topic
<code>cloud_topic</code>	<code>/camera/camera/depth/color/points</code>	PointCloud2 topic
<code>camera_info_topic</code>	<code>/camera/camera/color/camera_info</code>	Camera intrinsics topic
<code>model_path</code>	<code>updated.pt</code>	Path to YOLO weights file
<code>conf_threshold</code>	<code>0.5</code>	Minimum YOLO confidence to keep a detection
<code>show_image</code>	<code>True</code>	Display annotated video window
<code>target_frame</code>	<code>base_link</code>	TF2 target frame for 3D points
<code>target_class</code>	<code>""</code>	Empty = all classes; set to restrict picks to one class

4.1 YOLODetector Helper Class

File: `src/perception/perception/yolo_detector.py`

A stateless wrapper around Ultralytics YOLO. It is instantiated once inside `DetectionNode.__init__()` and called on every frame.

`detect(image) → list[dict]`

Runs YOLO inference on an OpenCV BGR image. Returns one dict per detection with fields:

- `class_id` — integer YOLO class index
- `class_name` — string label (e.g., "apple", "plate")
- `confidence` — float 0–1
- `bbox` — [x1, y1, x2, y2] pixel coordinates
- `center` — [cx, cy] bbox center in pixels

`draw_detections(image, detections) → np.ndarray`

Draws green bounding boxes, red centroid dots, and confidence labels onto a copy of the image. Used by `DetectionNode` when `show_image=True`.

4.2 pixel_to_world Helpers

File: `src/perception/perception/pixel_to_world.py`

`get_centroid_from_cloud_bbox(cloud_msg, camera_info_msg, bbox) → PointStamped | None`

Converts a YOLO bounding box into a 3D point in the camera frame by filtering the PointCloud2.

Steps:

1. Read camera intrinsics f_x , f_y , c_x , c_y from CameraInfo.K matrix
2. Parse PointCloud2 data as float32 array, shape (N, point_step//4)
Extract XYZ = first 3 floats per point
3. Drop NaN points and points with $z \leq 0$
4. Project each valid 3D point to image coords:
 $u = f_x * X/Z + c_x$
 $v = f_y * Y/Z + c_y$
5. Keep only points (u, v) that fall inside the YOLO bbox
6. Return mean(X), mean(Y), mean(Z) of filtered points as PointStamped
7. Return None if no valid points project into the bbox

transform_point(tf_buffer, point_stamped, target_frame) → PointStamped

A thin wrapper: calls `tf_buffer.lookup_transform(target_frame, source_frame, Time())` then `do_transform_point()`. Converts camera-frame points to base_link.

5. Node Deep-Dive: ConstantTransformPublisher

File: src/planning/planning/static_tf_transform.py | **Node name:** constant_tf_publisher

Broadcasts a single static TF transform on startup: **wrist_3_link** → **camera_link**.

This transform encodes the physical camera mount geometry:

- **Translation:** $x = -0.025$ m (slightly behind wrist), $y = +0.130$ m (to the side), $z = 0.0$ m
- **Rotation:** encoded in the 3×3 matrix $G[:3,:3]$ which reorders axes so the camera optical frame aligns correctly with the robot wrist frame

Why This Matters

Without this transform, TF2 cannot convert PointCloud2 depth data from camera_optical_frame into base_link. The DetectionNode calls transform_point() which performs a TF2 lookup through the chain: camera_depth_optical_frame → camera_link → wrist_3_link → ... → base_link. The static broadcaster must start before any detection is attempted.

Implementation Detail

The rotation matrix $G[:3,:3]$ is converted to a quaternion using `scipy Rotation.from_matrix().as_quat()` and stored in a TransformStamped message published via StaticTransformBroadcaster. The transform is sent once in `__init__()` — there is no timer or callback.

■ The `child_frame_id` is `camera_link` (the ROOT of the RealSense TF sub-tree), NOT an optical frame. Publishing to an optical frame would create a parent conflict — two nodes would both claim to be the parent of that frame, breaking TF2.

6. Node Deep-Dive: IKPlanner

File: src/planning/planning/ik.py | **Node name:** ik_planner

IKPlanner is a ROS2 node that provides two synchronous blocking calls used by both ArmCircler and UR7e_CubeGrasp. It communicates with MoveIt 2 via service calls.

compute_ik(current_joint_state, x, y, z, qx=0, qy=1, qz=0, qw=0) → JointState | None

Requests MoveIt inverse kinematics for a Cartesian target. The default quaternion (qy=1, all others 0) represents a 180° rotation around Y — this is the gripper-pointing-straight-down orientation. It is the correct default for picking objects off a table.

Internal steps:

1. Build PoseStamped in frame "base_link" with (x, y, z) + quaternion
2. Fill GetPositionIK.Request:
ik_link_name = "tool0" (the gripper link)
pose_stamped = target pose
robot_state.joint_state = current_joint_state (seed for IK solver)
avoid_collisions = True
timeout = Duration(sec=5)
group_name = "ur_manipulator"
3. Call /compute_ik service (synchronous – blocks until response)
4. If error_code == SUCCESS: return result.solution.joint_state
Otherwise: return None

Critical: each IK call uses the PREVIOUS step's joint solution as the seed. This keeps the IK solver in a locally consistent joint-space region and prevents elbow flips between adjacent waypoints.

plan_to_joints(target_joint_state, start_joint_state=None) → RobotTrajectory | None

Calls MoveIt motion planning (RRTConnect) to generate a collision-free trajectory from the current state to a target joint configuration.

Internal steps:

1. Build GetMotionPlan.Request:
group_name = "ur_manipulator"
allowed_planning_time = 10.0 s
planner_id = "RRTConnectkConfigDefault"
start_state.joint_state = start_joint_state (if provided)
2. For each joint in target_joint_state: add JointConstraint
tolerance_above = tolerance_below = 0.005 rad
weight = 1.0
3. Call /plan_kinematic_path service (synchronous)
4. Return motion_plan_response.trajectory on SUCCESS, else None

Startup Wait

In `__init__()`, IKPlanner blocks in a loop calling `wait_for_service()` on both `/compute_ik` and `/plan_kinematic_path` until they are available. This ensures MoveIt is fully started before any IK is

attempted.

7. Node Deep-Dive: ArmCircler

File: src/planning/planning/arm_circler.py | **Node name:** arm_circler

ArmCircler drives the arm in a 2-row orbital path around the plate to capture images from multiple viewpoints. It is enabled by default and disabled with `skip_circler:=true`.

Subscriptions

- **/detected_plate_point** → `_plate_callback(msg)` — triggers orbit construction (once only, guarded by `_orbit_triggered` flag)
- **/joint_states** → `_joint_state_callback(msg)` — stores latest joint state for IK seeding
- **/camera/camera/color/image_raw** → `_photo_callback(msg)` — buffers latest camera frame

Publishers

- **/orbit_done** (Bool, TRANSIENT_LOCAL) — published once when all waypoints complete
- **/start_pick_place** (Bool) — also available as publisher (unused in current flow; commander handles this)

plate_callback() — Orbit Trigger

This is the main initialization callback. It runs exactly once (guarded by `self._orbit_triggered`). On first call:

```

1. Set _orbit_triggered = True (prevents re-triggering)
2. Clean and recreate captured_images/ directory
3. Extract plate centroid (cx, cy, cz) from message
4. Build orbit geometry:
radius = 0.15 m (horizontal distance from plate centroid)
height = 0.30 m (row 1) / 0.20 m (row 2)
num_points = 20 per row (40 total)
tilt_distance = 0.12 m (row 1) / 0.17 m (row 2)

5. For each of 2 rows:
angles = linspace(- $\pi/4$ ,  $\pi + \pi/8$ , 20)
Row 2 angles are REVERSED so the arm sweeps back
For each angle  $\theta$ :
tx = cx + radius * cos( $\theta$ )
ty = cy + radius * sin( $\theta$ )
tz = cz + height

Orientation: look-at rotation pointing back toward plate
rot_z = R.from_euler("z",  $\theta - \pi/2$ ) # face inward
rot_y = R.from_euler("y",  $\pi$ ) # flip to point down
tilt_angle = arctan2(radius + tilt_distance, height)
rot_x = R.from_euler("x", tilt_angle) # tilt toward plate
combined = rot_z * rot_y * rot_x
q = combined.as_quat() # [x, y, z, w]

Call compute_ik(joint_state, tx, ty, tz, qx, qy, qz, qw)
If IK succeeds: append JointState to job_queue
6. After both rows: add a final "home-like" pose above the plate
7. Save pose_data as poses.npy (shape: (41, 7) including plate centroid)
8. Call _execute_jobs() to start sequential execution

```

_execute_jobs() — Sequential Waypoint Execution

Pops one JointState from job_queue, calls plan_to_joints(), then calls _execute_joint_trajectory(). After each trajectory completes:

- `_on_exec_done()` fires → calls `_take_photo()` to save the current frame to `captured_images/`
- Then calls `_execute_jobs()` recursively for the next waypoint
- When queue is empty: publishes `Bool(True)` on `/orbit_done`

Note: `_take_photo()` silently skips if `self._frame` is `None` (no camera frame buffered yet). Images are saved as `captured_image_1.jpg` through `captured_image_N.jpg`.

poses.npy Format

The first entry in `poses.npy` is the plate centroid `[cx, cy, cz, 0, 0, 0, 1]`. Entries 1..N are the arm end-effector poses at each waypoint: `[tx, ty, tz, qx, qy, qz, qw]`. Pose `i` corresponds to `captured_image_{i}.jpg` (0-indexed in `poses.npy`, 1-indexed in filenames).

8. Node Deep-Dive: UR7e_CubeGrasp (main.py)

File: src/planning/planning/main.py | Node name: cube_grasp

This is the central execution node. It owns the **busy** semaphore, the **job_queue**, and all state machine flags. It interfaces with IKPlanner, the trajectory controller, and the gripper service.

8.1 Subscriptions and Publishers

Subscriptions

- **/detected_pick_point** (PointStamped) → `cube_callback()` — main pick trigger
- **/detected_class** (String) → `class_callback()` — stores `self.detected_class` for offset lookup
- **/joint_states** (JointState) → `joint_state_callback()` — stores latest joints; triggers initial `_go_home()` on first message
- **/detected_plate_point** (PointStamped) → `plate_callback()` — stores `self.plate_pose` as default drop target
- **/pick_task_list** (String/JSON) → `_on_task_list()` — loads pre-computed task list
- **/start_pick_place** (Bool) → `_on_start_pick_place()` — activates pick-and-place mode
- **/target_drop_point** (PointStamped) → `_on_target_drop()` — sets drop override + arms `_pick_triggered`

Publishers

- **/orbit_done** (Bool, TRANSIENT_LOCAL) — published in `__init__()` when `skip_circler=True`

Action Clients

- **/scaled_joint_trajectory_controller/follow_joint_trajectory** (FollowJointTrajectory) — executes planned trajectories on the robot

Service Clients

- **/toggle_gripper** (Trigger) — opens/closes gripper (toggle; state tracked by `self.gripper_open`)

8.2 Job Queue Mechanism

The job queue (`self.job_queue`) is a flat list where each entry is one of:

- **JointState** — a target joint configuration; calls `plan_to_joints()` then `_execute_joint_trajectory()`
- **'toggle_grip'** (string literal) — calls `_toggle_gripper()` synchronously

`execute_jobs()` pops the front of the queue and dispatches the job. Each async completion callback calls `execute_jobs()` again. This creates a chain: the ROS event loop is never blocked — each job fires the next one via callback.

Example job queue for a pick-place cycle (after `_on_refined_detection`):

```
[0] JointState → pre_grasp_joints (hover above object)
[1] JointState → grasp_joints (descend to grasp height)
[2] "toggle_grip" (close gripper)
[3] JointState → lift_joints (lift object)
[4] JointState → drop_pre_joints (move above drop zone)
[5] JointState → drop_joints (descend to release height)
[6] "toggle_grip" (open gripper)
```

After queue drains: `execute_jobs()` detects empty queue, `going_home=False`
→ calls `_go_home()` → appends home JointState → clears busy when home reached

8.3 Two-Phase Centroid Refinement

Detections at the camera's field-of-view edge are distorted. The two-phase system corrects this before committing to the full grasp trajectory:

PHASE 1 (`cube_callback` – initial detection):

- Compute IK for $(cx, cy, cz + 0.5)$ -- 0.5 m directly above rough centroid
- Append only this one waypoint to `job_queue`
- Set `_refining = True`
- `execute_jobs()` runs, arm moves to pre-pregrasp

`execute_jobs()` sees empty queue while `_refining == True`:

- Sets `_at_pre_pregrasp = True`
- Prints "Pre-pregrasp reached. Waiting for refined centroid..."
- Returns (waits for next detection)

PHASE 2 (`cube_callback` – refined detection):

- `cube_callback` checks: `_refining AND _at_pre_pregrasp` → calls `_on_refined_detection()`
- Uses fresh, well-centered centroid with full grasp offsets
- Appends 7 jobs: `pre_grasp`, `grasp`, `toggle`, `lift`, `drop_pre`, `drop`, `toggle`
- `execute_jobs()` continues the sequence

8.4 Task-List Auto-Execution

When a pre-computed task list is loaded via `/pick_task_list`, the system executes picks automatically without human input. The flow after `/start_pick_place` is received:


```

_on_start_pick_place() called:
→ sets pick_place_enabled = True
→ calls _go_home() (sets busy=True)

Home reached → execute_jobs() sees empty queue, _going_home=True:
→ _going_home = False, busy = False
→ checks: self._task_list and self._task_idx < len(_task_list)
→ calls _start_next_task()

_start_next_task():
→ pops task[_task_idx] (object_name, pick:[x,y,z], drop:[x,y,z])
→ sets self.detected_class = object_name
→ sets self._current_drop_override = [drop_x, drop_y, drop_z]
→ computes IK for (cx, cy, cz + 0.5) (pre-pregrasp above 6D estimate)
→ sets _refining = True, appends pre-pregrasp JointState
→ execute_jobs() → arm moves to pre-pregrasp

PHASE 2 fires when live detection sees the object below the arm:
→ _on_refined_detection() uses _current_drop_override as drop position
→ executes full pick-place sequence

After drop, job queue drains → _go_home() → home reached:
→ _start_next_task() for next item, or "All tasks complete."

```

8.5 State Machine: busy / _refining / _going_home

Flag	Type	True Means	Cleared By
busy	bool	Arm is in motion or a pick cycle is active; cube_callback() returns immediately	execute_jobs() when home is reached; also cleared on trajectory failure
_refining	bool	Arm is moving to or waiting at pre-pregrasp; subsequent cube_callback() detections are suppressed	Set False in _on_refined_detection() when Phase 2 fires
_at_pre_pregrasp	bool	Arm has physically arrived at pre-pregrasp; next cube_callback() should call _on_refined_detection()	Set False in _on_refined_detection()
_going_home	bool	Home waypoint is in the job queue (or arm is moving home)	Set False when execute_jobs() sees empty queue and _going_home=True
pick_place_enabled	bool	System will respond to detections; False until /start_pick_place or skip_circlear=true	Never reset; permanently True once activated
_pick_triggered	bool	One live-detection pick has been armed by /target_drop_point	Cleared in cube_callback() when pick is accepted

8.6 Pick Sequence — Full Call Chain

A complete pick cycle from first detection to home, showing every function call:

```
image_callback() [DetectionNode, ~30 Hz]
■■ detect() → transform_point() → publish /detected_class, /detected_pick_point
```

```
cube_callback(pick_pose) [UR7e_CubeGrasp]
■■ check: pick_place_enabled, _pick_triggered, not busy, joint_state, plate_pose
■■ _pick_triggered = False (consume one-shot)
■■ busy = True
■■ compute_ik(joint_state, cx, cy, cz+0.5)
■■ _refining = True
■■ job_queue.append(pre_pre_grasp_joints)
■■ execute_jobs()
■■ plan_to_joints() → _execute_joint_trajectory()
■■ exec_ac.send_goal_async() → _on_goal_sent()
■■ goal_handle.get_result_async() → _on_exec_done()
■■ execute_jobs() [queue now empty, _refining=True]
■■ _at_pre_pregrasp = True (WAIT)
```

... arm physically at pre-pregrasp, next detection fires ...

```
cube_callback(refined_pose) [_refining AND _at_pre_pregrasp]
■■ _on_refined_detection(refined_pose)
■■ _refining = False, _at_pre_pregrasp = False
■■ drop from _current_drop_override (task-list) or plate_pose
■■ lookup offsets in PICK_OFFSETS[detected_class]
■■ compute_ik(joint_state, cx+x_off, cy+y_off, cz+pre_grasp_z)
■■ compute_ik(pre_grasp, cx+x_off, cy+y_off, cz+grasp_z)
■■ compute_ik(grasp, cx+x_off, cy+y_off, cz+lift_z)
■■ compute_ik(lift, drop_x, drop_y, drop_z+0.20)
■■ compute_ik(drop_pre, drop_x, drop_y, drop_z+0.15)
■■ job_queue = [pre_grasp, grasp, "toggle_grip",
lift, drop_pre, drop, "toggle_grip"]
■■ execute_jobs() → runs 7 jobs sequentially via callbacks
```

```
After job 7: execute_jobs() → _going_home=False → _go_home()
■■ busy=True, _going_home=True, job_queue=[home_joints]
■■ execute_jobs() → plan → execute → _on_exec_done()
■■ execute_jobs(): queue empty, _going_home=True
→ _going_home=False, busy=False
→ if task list: _start_next_task()
→ else: "Home reached. Ready for next pick command."
```

9. Node Deep-Dive: Commander

File: src/planning/planning/commander.py | **Node name:** commander

Commander is the high-level orchestrator. It runs in a separate terminal (stdin is required for user interaction). It receives the orbit-done signal, runs the two-stage pose pipeline as subprocesses, presents results to the user, and arms main.py for individual picks.

Subscriptions

- **/orbit_done** (Bool, TRANSIENT_LOCAL) → `_on_orbit_done()` — triggers pose pipeline in a background thread
- **/detected_plate_point** (PointStamped) → `plate_callback()` — stores plate pose for drop position fallback

Publishers

- **/start_pick_place** (Bool) — published when user presses Enter
- **/set_target_class** (String) — published with each pick command to switch YOLO target
- **/target_drop_point** (PointStamped) — published with each pick command; receiving this also arms main.py's `_pick_triggered`

`_on_orbit_done()` → `_run_pose_pipeline()` [background thread]

When `/orbit_done` fires, Commander spawns a daemon thread to run the full pose pipeline. This keeps the ROS spin loop responsive:

```
Thread: _run_pose_pipeline()
cwd = os.getcwd() (must be lab5/ for path resolution)

Step 1: YOLO + SAM2 segmentation subprocess
cmd = [SAM2_PYTHON, segment_batch.py,
--input_dir captured_images/
--output_dir segmented/,
--yolo updated.pt]
subprocess.run(cmd, check=True)
Output: segmented/captured_image_N/<stem>_<class>_<i>_mask.png

Step 2: 6D pose estimation subprocess
cmd = [SAM2_PYTHON, -c, "
import sys, os; sys.path.insert(0, SIXD_DIR); os.chdir(SIXD_DIR);
from run import run_pose_estimation;
run_pose_estimation(use_multi_view=True, position_only=True, ...)"]
Output: results/object_poses.npy

Step 3: Load and display results
_pose_results = np.load("results/object_poses.npy", allow_pickle=True)
_print_objects(): prints [idx] name pos=(x,y,z) conf=0.xx

Step 4: _prompt_user() — blocks on stdin
```

Path Resolution

Commander uses `_find_project_root()` to locate the project by walking up the directory tree until it finds a parent containing an **armcircular/** subdirectory. This works whether running from source or from the installed package (which can be 7+ levels deep inside install/).

```
_PROJECT_ROOT = <parent containing armcircular/>
_SIXD_DIR = _PROJECT_ROOT/lab5/src/planning/planning/6D_poses/
_SEG_SCRIPT = _PROJECT_ROOT/lab5/segment_batch.py
_YOLO_WEIGHTS = _PROJECT_ROOT/lab5/src/planning/planning/updated.pt
_SAM2_PYTHON = _PROJECT_ROOT/sam2/sam2_env/bin/python (venv with sam2+torch)
```

`_prompt_user()` — Interactive CLI Loop

```
1. Blocks on input("Press Enter to activate pick-and-place: ")
→ publishes Bool(True) on /start_pick_place
→ prints detected objects table

2. Enters while rclpy.ok() loop:
Prompts: "Type class name or index to pick (or list / q): "

Input is a digit:
→ look up _pose_results[idx]
→ _arm_for_pick(object_name, position_base_link_m)

Input is a string (class name):
→ search _pose_results for matching object_name
→ _arm_for_pick(user_in, drop_pos_from_results)

"list" → re-print detected objects
"q" → break loop

_arm_for_pick(class_name, drop_pos_arr):
1. Resolve drop position (pose results or plate centroid)
2. Publish String(class_name) on /set_target_class
3. Publish PointStamped(drop_x, drop_y, drop_z) on /target_drop_point
(receiving /target_drop_point in main.py sets _pick_triggered=True)
```

10. 6D Pose Estimation Pipeline

The 6D pose pipeline runs entirely offline between the orbit phase and the pick phase. It is invoked as a subprocess by Commander and outputs **results/object_poses.npy**. All files live in **src/planning/planning/6D_poses/**.

10.1 run.py — Entry Point

`run_pose_estimation()` is the top-level function called by Commander. It orchestrates the full pipeline:

```
run_pose_estimation(use_multi_view=True, position_only=True, ...)

1. Build PoseEstimator(configs=OBJECTS, position_only=True, ...)
   → position_only=True skips DINOv2 + STL rendering (faster)

2. estimator.build_databases(force_rebuild=False)
   → position_only: prints "skipping reference database build"

3. Load arm poses from poses.npy:
   _load_all_poses(poses_path) → list of 4x4 ee_T_world matrices
   MAX_IMAGES = 20 (YOLO reliable only on first 20 views)

4. For each of indices [0..19]:
   Load captured_image_{i+1}.jpg → raw_images[i]
   Build cam_T_world[i] = cam_T_world_from_ee(ee_T_world[i])
   = inv(T_CAM_FROM_EE) @ ee_T_world

5. estimator.process_multi_view(
   list(zip(raw_images, cam_poses)),
   mask_dirs = [segmented/captured_image_{i+1} for i in indices]
) → results list

6. _add_base_link_poses(results, cam_poses[0]):
   → for each result: position_base_link_m = triangulated_position_m
   (already in world/base_link frame from triangulation)

7. Save results/object_poses.npy:
   [{object_name, instance_idx, position_base_link_m, confidence}, ...]
```

Pose Transform Chain

Converting arm EE poses into camera frame poses:

```
_pose7_to_ee_T_world(p):
p = [x, y, z, qx, qy, qz, qw] (arm EE pose in world/base_link frame)
world_T_ee = 4x4 matrix from quaternion + translation
ee_T_world = inv(world_T_ee)

cam_T_world_from_ee(ee_T_world):
T_CAM_FROM_EE = fixed 4x4 offset (camera mount: +3cm forward, -13.5cm above)
cam_T_world = inv(T_CAM_FROM_EE) @ ee_T_world
This converts world coordinates into the camera frame at each orbit position
```

10.2 pose_estimator.py — Five Sub-Components

Component 1: ReferenceRenderer

Renders an object's STL mesh from N viewpoints using pyrender + trimesh. Used only in full 6DoF mode (position_only=False).

- **load_mesh(cfg)** — loads STL, normalises to unit sphere, applies ObjectConfig.color_rgb as material
- **render_views(cfg, n_views=500)** — renders from 500 Fibonacci-sphere viewpoints; returns list of {image, depth, pose_w2c, pose_c2w} dicts
- **fibonacci_sphere(n)** — generates n evenly-spaced unit vectors on a sphere using the golden-ratio spiral method

Component 2: DinoFeatureExtractor

Wraps DINOv2 ViT-S/14 (from Facebook Research). Used only in full 6DoF mode.

- **extract(image, mask=None)** — preprocesses to 224×224, normalises, runs DINOv2 forward pass, returns {cls: CLS token, patches: patch tokens}
- Mask zeroes out background pixels before feature extraction, improving discriminability

Component 3: ReferenceDatabase

Stores 500 rendered views + their DINOv2 CLS features for KNN retrieval. Used only in full 6DoF mode.

- **build(views, extractor)** — extracts features for all views, builds KNN index (cosine similarity, brute-force)
- **query(cls_vec, k=5)** — L2-normalises query, returns k nearest reference views with their w2c poses
- **save/load(path)** — caches to .npz to avoid recomputing

Component 4: PoseRefiner

Iterative render-and-compare edge-based refinement. Used only in full 6DoF mode.

- **refine(coarse_pose_w2c, real_crop, n_iterations=6)** — starts from coarse DINO pose; iteratively perturbs and re-renders; keeps best by edge_score
- **edge_score(render, real_crop)** — Chamfer-like distance on Canny edges: (render_edges * dist_transform(real_edges)).sum() + symmetric
- **perturb(pose, n=12)** — random rotation perturbations (skipped for symmetric objects) + random translation perturbations
- **resolve_symmetry(cfg)** — if cfg.is_symmetric is None, calls _detect_sphere() on the STL to auto-detect rotationally symmetric objects

Component 5: PoseEstimator (top-level API)

The main entry point for pose estimation. Owns databases and refiners.

process_multi_view() — used in current pipeline (position_only=True)

```

process_multi_view(views, mask_dirs, seg_name_map, fx, fy, ...)
position_only=True → delegates to _multi_view_position_only()

_multi_view_position_only(views, mask_dirs, seg_name_map, fx, fy, cx, cy):
For each view i:
all_masks = load_masks_from_dir(mask_dirs[i], seg_name_map)
→ parses filenames: <stem>_<seg_label>_<inst_idx>_mask.png
→ returns {config_name: [binary_mask0, mask1, ...]}
_scene_centroids_only(image, all_masks, fx, fy):
For each object mask:
u_c = median(x pixels in mask) (robust to edge clipping)
v_c = median(y pixels in mask)
area = mask pixel count
depth estimation (if diameter_m known):
r_px = sqrt(area /  $\pi$ ) (effective radius in pixels)
z_m = fx * (diameter_m/2) / r_px
x_m = (u_c - cx) / fx * z_m
y_m = (v_c - cy) / fy * z_m
position_m = [x_m, y_m, z_m] (camera frame)
centroid_uv = (u_c, v_c)
mask_area = area
Returns list of {object_name, instance_idx, centroid_uv, mask_area, position_m}

Group observations by (object_name, instance_idx)

For each group:
triangulate_ransac(uvs, cam_poses, areas, fx, fy, cx, cy)
→ triangulated_position_m (in world/base_link frame)
confidence = min(1.0, n_views_seen / n_total_views)

```

10.3 RANSAC + Weighted DLT Triangulation

The triangulation functions in pose_estimator.py recover the 3D world position of an object from 2D centroid observations across multiple camera views.

triangulate_dlt(centroids_uv, poses_w2c, fx, fy, cx, cy, weights)

```

Direct Linear Transform (DLT) triangulation:

For each view (u, v) with camera matrix P = K @ T_w2c:
Add 2 rows to matrix A (weighted by sqrt(mask_area)):
(u*P[2] - P[0]) * w
(v*P[2] - P[1]) * w

Solve: SVD(A) → take last right singular vector V[-1]
Return X = V[-1][:3] / V[-1][3] (homogeneous to 3D)

Weights = mask pixel counts → larger, clearer views have more influence

```

triangulate_ransac(centroids_uv, poses_w2c, mask_areas, fx, fy, cx, cy, n_iter=100)

```

RANSAC wrapper around DLT:
1. For n_iter iterations:
  Pick random pair (i, j) of views
  DLT triangulate from just those 2 views → X_candidate
  For each view k: reproject X_candidate, measure pixel error
  Count inliers (reprojection error < 8 px)
  Keep best inlier set

```

```

2. If best_inliers < 2: use all views (fallback)

```

```

3. Re-run DLT on inlier views only, weighted by mask_area
  → final triangulated_position_m (world frame)

```

```

Returns (X_world, inlier_indices)

```

Effect: rejects views where the object is partially occluded,
at glancing angle, or where the mask centroid is unreliable.

10.4 constants.py — Configuration

Central configuration file for the 6D pose pipeline. Key entries:

Object Name	diameter_m	color_rgb	is_symmetric
apple	0.045 m	(200, 40, 40)	auto (sphere)
blueberry	0.015 m	(60, 40, 100)	auto (sphere)
cherry	0.020 m	(180, 30, 30)	auto (sphere)
cake	0.075 m	(80, 40, 15)	False (asymmetric)
grape	0.0236 m	(100, 50, 120)	auto
half_grape	0.0236 m	(100, 50, 120)	False
halved_strawberry	0.0301 m	(210, 50, 60)	False
mango_piece	0.040 m	(220, 150, 50)	False
small_tomato	0.030 m	(210, 50, 40)	auto
strawberry	0.0301 m	(210, 50, 60)	False

diameter_m directly affects depth accuracy: $z = f_x \times (\text{diameter}/2) / r_{\text{px}}$. Wrong diameters lead to wrong z estimates and missed picks. Tune per-object against real measurements.

```

SEG_NAME_MAP: maps YOLO/SAM2 label names in mask filenames → ObjectConfig names
"apple" → "apple"
"blueberry" → "blueberry"
"cake" → "cake"
"grape" → "grape"
"strawberry" → "strawberry"
"tomato" → "small_tomato"
Labels absent from this map are skipped by pose estimation.

```


11. End-to-End Pipeline Call Chain

This section traces the complete system from startup to completing all picks, showing which node calls what.

Stage 1: Startup

```
Terminal 1:
ros2 launch realsense2_camera rs_launch.py pointcloud.enable:=true ...
→ Starts /camera/... topics

Terminal 2:
ros2 launch planning lab5_bringup.launch.py robot_ip:=... target_class:=apple
→ Starts: ConstantTransformPublisher (broadcasts wrist_3_link→camera_link TF)
→ Starts: IKPlanner (waits for /compute_ik, /plan_kinematic_path)
→ Starts: DetectionNode (starts YOLO inference loop)
→ Starts: ArmCircler (subscribes to /detected_plate_point)
→ Starts: UR7e_CubeGrasp
joint_state_callback() fires on first /joint_states message
→ _go_home() called (arm moves to observation pose)

Terminal 3:
cd lab5/
ros2 run planning commander
→ Commander subscribes to /orbit_done (TRANSIENT_LOCAL – will get it late)
→ Commander subscribes to /detected_plate_point
→ Prints "Waiting for orbit to complete (/orbit_done)..."
```

Stage 2: Plate Detection Triggers Orbit

```
DetectionNode.image_callback():
→ YOLO detects "plate" in frame
→ get_centroid_from_cloud_bbox() → 3D centroid
→ transform_point() → base_link frame
→ publish /detected_plate_point (PointStamped)

ArmCircler._plate_callback(msg):
→ First call only (_orbit_triggered guard)
→ Delete + recreate captured_images/ directory
→ Build 40 waypoints (2 rows × 20 angles) around plate centroid
→ For each waypoint: compute_ik() → append to job_queue
→ Save poses.npy
→ Call _execute_jobs()
Loop: plan_to_joints() → execute → photo → next
→ When queue empty: publish Bool(True) on /orbit_done
```

Stage 3: Offline Pose Pipeline (Commander)

```

Commander._on_orbit_done(msg):
→ Spawn daemon thread: _run_pose_pipeline()

Thread _run_pose_pipeline():
→ subprocess: SAM2_PYTHON segment_batch.py
Reads: captured_images/captured_image_{1..N}.jpg
Runs: YOLO detection + SAM2 instance segmentation
Writes: segmented/captured_image_N/<stem>_<class>_<i>_mask.png

→ subprocess: SAM2_PYTHON -c "from run import run_pose_estimation; ..."
Reads: poses.npy, captured_images/, segmented/
Runs: _load_all_poses() → cam_T_world per view
process_multi_view() → _multi_view_position_only()
→ per view: load_masks_from_dir()
_scene_centroids_only() → centroid_uv, depth
triangulate_ransac() → position_base_link_m per object
Writes: results/object_poses.npy

→ Load results, print table
→ Block on: input("Press Enter to activate pick-and-place: ")

```

Stage 4: User Activates + Pick Loop

```

User presses Enter:
Commander._prompt_user():
→ publish Bool(True) on /start_pick_place

UR7e_CubeGrasp._on_start_pick_place():
→ pick_place_enabled = True
→ _go_home() [if task list: "N tasks queued. Going home first."]

Home reached → execute_jobs() → _start_next_task()
OR
User types "2" in Commander:
Commander._arm_for_pick("cake", pose_results[2]["position_base_link_m"])
→ publish String("cake") on /set_target_class
→ publish PointStamped(drop_x, drop_y, drop_z) on /target_drop_point

DetectionNode._set_target_class_cb(): target_class = "cake"

UR7e_CubeGrasp._on_target_drop(): _current_drop_override = [x,y,z], _pick_triggered =
True

Next DetectionNode frame with "cake" detected:
→ publish /detected_class("cake"), /detected_pick_point(pt)

UR7e_CubeGrasp.cube_callback(pt):
[Phase 1 → Phase 2 → pick-place cycle as described in Section 8.6]

After pick: arm returns home, ready for next command.

```

12. Per-Class Grasp Offsets Reference

These offsets are applied to the **refined centroid** (Phase 2) in `_on_refined_detection()`. All values are in metres. The x/y offsets correct for systematic camera-to-gripper misalignment. The z offsets define the grasp trajectory heights.

Class	x_offset	y_offset	pre_grasp_z	grasp_z	lift_z
apple	0.010	0.020	0.160	0.130	0.185
tomato	0.015	0.025	0.160	0.145	0.185
cake	0.015	0.025	0.200	0.140	0.200
strawberry	0.020	0.020	0.200	0.145	0.200
cherry	0.020	0.020	0.200	0.140	0.200
grape	0.020	0.020	0.200	0.144	0.200
blueberry	0.020	0.020	0.200	0.145	0.200
chocolate	0.020	0.005	0.200	0.143	0.200
DEFAULT	0.020	0.005	0.160	0.140	0.185

How these are applied (in `_on_refined_detection()`):

```
cx, cy, cz = refined centroid in base_link
```

```
pre_grasp = (cx + x_offset, cy + y_offset, cz + pre_grasp_z) ← hover
```

```
grasp = (cx + x_offset, cy + y_offset, cz + grasp_z) ← close gripper
```

```
lift = (cx + x_offset, cy + y_offset, cz + lift_z) ← lift
```

```
drop_pre = (drop_x, drop_y, drop_z + 0.20) ← above plate
```

```
drop = (drop_x, drop_y, drop_z + 0.15) ← release
```

Tuning guide:

grasp_z too high → gripper closes on air above object

grasp_z too low → gripper hits table surface

x/y_offset wrong → gripper center misses object

13. Key Design Patterns

Pattern 1: busy Semaphore

The `self.busy` boolean in `UR7e_CubeGrasp` prevents concurrent pick cycles. Once a pick is accepted in `cube_callback()`, `busy` is set `True` and all subsequent `cube_callback()` calls return immediately. It is only cleared after the arm returns home (or on trajectory failure). This guarantees exactly one pick cycle executes at a time.

Pattern 2: IK Chaining (Seed Propagation)

Each `compute_ik()` call in the pick sequence uses the *previous step's joint solution* as the seed, not the live `/joint_states`. This is critical: it keeps the IK solver exploring solutions in the same joint-space neighborhood, preventing the arm from making large, unexpected elbow flips between adjacent waypoints.

```
seed = self.joint_state (current arm position)
pre_grasp = compute_ik(seed, ...)
grasp = compute_ik(pre_grasp, ...)
lift = compute_ik(grasp, ...)
drop_pre = compute_ik(lift, ...)
drop = compute_ik(drop_pre, ...)
```

Pattern 3: Async Trajectory Execution via Callbacks

Trajectories are executed without blocking the ROS event loop:

```
execute_jobs()
■ ■ _execute_joint_trajectory(joint_traj)
■ ■ exec_ac.send_goal_async(goal)
■ ■ callback: _on_goal_sent(future)
■ ■ goal_handle.get_result_async()
■ ■ callback: _on_exec_done(future)
■ ■ execute_jobs() [next job]
```

The ROS spin loop is never stalled. The arm executes while ROS keeps processing subscriptions (so detection topics stay live).

Pattern 4: TRANSIENT_LOCAL QoS for One-Time Signals

`/orbit_done` is published once and must be received by Commander even if Commander starts late. `TRANSIENT_LOCAL` (latched) QoS ensures any subscriber that joins after the message is published will still receive the last value. Both publisher and subscriber must use `TRANSIENT_LOCAL` — if they mismatch, the subscriber gets nothing.

Pattern 5: Two-Phase Centroid Refinement

The initial YOLO centroid may be distorted if the object is near the camera's field-of-view edge. By first moving directly above the rough estimate (pre-pregrasp), the camera gets a near-overhead, undistorted view. The refined centroid from this position is used for the actual grasp trajectory. This corrects systematic radial distortion without any calibration data.

Pattern 6: Topic Ordering Race Condition

/detected_class MUST arrive before /detected_pick_point in UR7e_CubeGrasp. The class is used to select per-class offsets in PICK_OFFSETS. If /detected_pick_point arrives first, self.detected_class will be stale or empty and DEFAULT_OFFSETS will be used silently. DetectionNode always publishes class then pick_point in the same callback, so this is only a risk if network latency separates them.

Pattern 7: Background Subprocess Management in Commander

The YOLO+SAM2 segmentation and 6D pose estimation are run as external Python subprocesses using the sam2 virtual environment's Python interpreter. This is because these tools require packages (torch, sam2, trimesh, pyrender) that are not available in the ROS Python environment. Commander runs them via subprocess.run() in a daemon thread to keep the ROS node responsive.

Pattern 8: Graceful Failure Recovery

In _on_exec_done(), if the trajectory action returns an exception (e.g., controller rejected goal), the handler clears busy, job_queue, and all sentinel flags, then logs the error. This prevents the system from hanging indefinitely in a broken state. The next detection event can start a fresh pick cycle.

14. Known Issues and Watch-outs

Camera Launch

■ The RealSense camera is commented out in `lab5_bringup.launch.py`. It must be started in a separate terminal before `bringup`. Forgetting this will cause `DetectionNode` to never receive data (watchdog timer will warn after 5s).

IK Failure on Drop Waypoints

`drop_pre_joints` and `drop_joints` IK failures are not checked before enqueueing in `_on_refined_detection()`. If IK returns `None` for a drop waypoint, `None` will be passed to `plan_to_joints()`, which will attempt to iterate over `None.name` and raise an `AttributeError`. MoveIt IK failure on the drop zone usually means the plate is in an unreachable area — move the plate closer to the robot's workspace.

Gripper State Assumption

`self.gripper_open` is initialised to `True` in `UR7e_CubeGrasp`. If the gripper is physically closed at startup, the first `toggle_grip` will open it instead of close. The fix is to manually open the gripper before launching.

Object Moves After Pre-Pregrasp

After the arm reaches pre-pregrasp, the system waits indefinitely for a detection. If the object is removed or moves out of YOLO's field of view, the arm will remain at pre-pregrasp forever. `busy` stays `True`. A timeout mechanism could be added to `execute_jobs()` in the `_at_pre_pregrasp` wait state.

Commander Must Run in Separate Terminal

Commander blocks on `stdin` (`input()` calls). `stdin` is not forwarded to nodes launched via `ros2 launch`. Commander must always be started manually with `ros2 run planning commander` in its own terminal.

Source vs Install Sync

`colcon build` copies Python files into `install/planning/lib/python3.10/site-packages/planning/`. It does NOT create symlinks. Every change to `src/planning/planning/*.py` must be manually mirrored to the install tree until the next `colcon build`:

```
cp src/planning/planning/main.py
install/planning/lib/python3.10/site-packages/planning/
cp src/planning/planning/commander.py
install/planning/lib/python3.10/site-packages/planning/
cp src/planning/planning/arm_circler.py
install/planning/lib/python3.10/site-packages/planning/
```

6D Position Accuracy

The depth estimate $z = f_x \times (\text{diameter}_m/2) / r_{px}$ is sensitive to the `diameter_m` values in `constants.py`. A 10% error in diameter produces a roughly 10% error in z , which directly affects `grasp_z_offset` accuracy. Measure actual object diameters and update `constants.py` for best results.

busy=True During Go-Home After /start_pick_place

When the user presses Enter and pick-and-place activates, `_go_home()` is called immediately (sets `busy=True`). Any YOLO detection that fires during the home trajectory is ignored. Wait for "Home reached. Ready for next pick command." in the logs before expecting a pick to trigger.

Task List Consumes Picks Automatically

If a task list was published before `/start_pick_place`, `main.py` will automatically execute all tasks after going home — without any further Commander input. Typing class names in Commander after this only affects the live-detection fallback (used only when the task list is exhausted or empty).

End of Document — MEC106A Lab 5 Code Guide